

Student Information System

A console-based Student Information System built entirely with **core Java** — no external libraries, frameworks, or dependencies. It demonstrates OOP design, input validation, collections, and a clean layered architecture (model / business logic / utilities / UI).

Table of Contents

1. [Features](#)
 2. [Project Structure](#)
 3. [Installation Guide](#)
 4. [User Manual](#)
 5. [Code Explanation](#)
 6. [Validation Rules](#)
 7. [Known Limitations](#)
-

1. Features

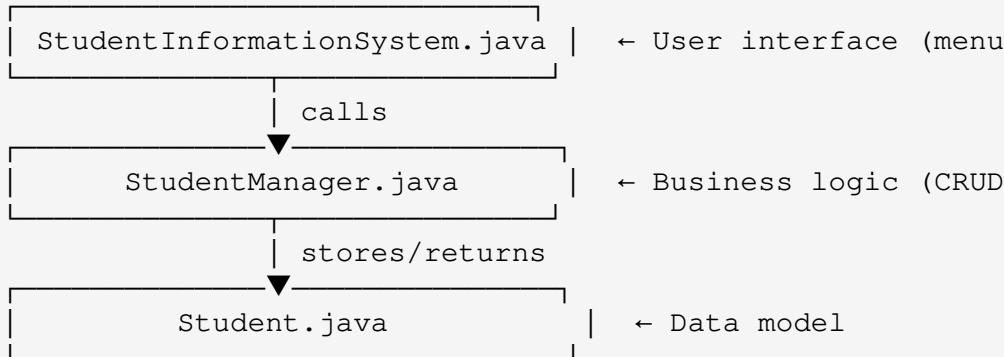
- Add, view, search, update, and delete student records
 - Fully in-memory (no database or file setup required to run)
-

2. Project Structure

```
StudentInformationSystem/
```

```
|— Student.java                → Data model (entity class)
|— StudentManager.java        → Business logic / data access
|— StudentInformationSystem.java → Main class with console UI
```

Layered design:



3. Installation Guide

Prerequisites

- **JDK 8 or later** installed (tested on JDK 21).
- Check your version:

```
java -version
javac -version
```

If these commands aren't recognized, install a JDK first (e.g., [Eclipse Temurin](#) or `sudo apt install default-jdk` on Ubuntu/Debian).

Steps

1. **Get the files.** Place all four `.java` files in the same folder:

```
Student.java
StudentManager.java
StudentInformationSystem.java
```

2. **Compile all files** from a terminal in that folder:

```
javac *.java
```

This produces four `.class` files. No errors should appear on a successful compile.

3. **Run the program:**

```
java StudentInformationSystem
```

4. **You should see:**

```
=====
      WELCOME TO THE STUDENT INFORMATION SYSTEM
=====

----- MAIN MENU -----
1. Add Student
2. View All Students
3. Search Student
4. Update Student
5. Exit
-----
Enter your choice:
```

No IDE is required — any text editor plus the JDK is enough. It also

works unmodified inside IntelliJ IDEA, Eclipse, or VS Code (Java Extension Pack) if you prefer an IDE.

4. User Manual

The system is menu-driven. Type a number (1–7) and press Enter to choose an action.

4.1 Add Student (Option 1)

You'll be prompted for each field in order. Invalid entries are rejected immediately with a message, and you'll be asked again — the program never crashes on bad input.

Field	Rule
ID	Positive whole number, must be unique
Name	Letters and spaces only, 2–50 characters
Age	Whole number between 5 and 100
Email	Standard format, e.g. <code>name@example.com</code>
Phone	10–13 digits, optional leading +
Marks	Number between 0 and 100

If the ID already exists, the add is cancelled with a warning.

4.2 View All Students (Option 2)

Prints every student on one line each, plus a total count. If no students exist yet, you'll see a message instead of an empty list.

4.3 Search Student (Option 3)

Enter part or all of a name (case-insensitive). All matching students are listed; if none match, you're told so.

4.4 Update Student (Option 4)

Enter the ID to update. If found, the current record is shown, then you re-enter **all** fields (name, age, email, phone, marks) with the same validation as adding — the ID itself cannot be changed.

4.5 Exit (Option 7)

Closes the input scanner and ends the program. **Note:** since data is stored in memory only, all records are lost on exit (see [Limitations](#)).

5. Code Explanation

5.1 `Student.java` — Data Model

A plain-old-Java-object (POJO) representing one student:

- **Fields:** `id`, `name`, `age`, `email`, `phone`, `marks`
- **Constructors:** a no-arg constructor and a full-argument constructor
- **Getters/Setters:** standard accessors for every field

5.2 `StudentManager.java` — Business Logic

Wraps an internal `ArrayList<Student>` and exposes operations without ever leaking the live list (methods like `getAllStudents()` return a **defensive copy** — `new ArrayList<>(students)` — so external code can't accidentally corrupt internal state).

Key methods:

- `addStudent()` — rejects duplicate IDs before inserting
- `searchStudent()` — case-insensitive partial match using `String.contains`
- `updateStudent()` — locates by ID, then overwrites fields in place
- `viewAllStudents()` — view all students from list

5.3 StudentInformationSystem.java — Entry Point / UI

Contains `main()`, a single `Scanner` instance shared across the whole session, and a `StudentManager` instance holding all data for the run.

- **Menu loop:** a `while (running)` loop paired with a Java 14+ **switch expression** (`switch (choice) { case 1 -> ...; }`) dispatches to one private method per menu option
- Every business action (`addStudent()`, `updateStudent()`, etc. — the private methods in this file) is a thin layer that only handles **console I/O**; it immediately delegates the actual data work to `StudentManager`

This keeps concerns cleanly separated:

`StudentInformationSystem` never touches the `ArrayList` directly, and `StudentManager` never calls `Scanner` or `System.out`.

6. Validation Rules (Reference)

Field	Rule	Example valid	Example invalid
ID	Positive integer	101	-5, 0
Name	2–50 letters/spaces	Alice Smith	Allce, `` (empty)
Age	5–100	20	3, 150

Email	local@domain.tld	alice@example.com	alice.com
Phone	10–13 digits, optional +	9876543210	12345
Marks	0–100	85.5	–10, 105

7. Known Limitations

- **No persistence:** all data lives in memory (`ArrayList`) and is lost when the program exits.
- **Single-user, single-session:** no concurrency handling or multi-user support.
- **Linear search:** `findById/searchByName` are $O(n)$ — fine for classroom-scale data, not optimized for very large datasets.
- **No file/database layer:** by design, to keep the project strictly "core Java" with zero setup.

--